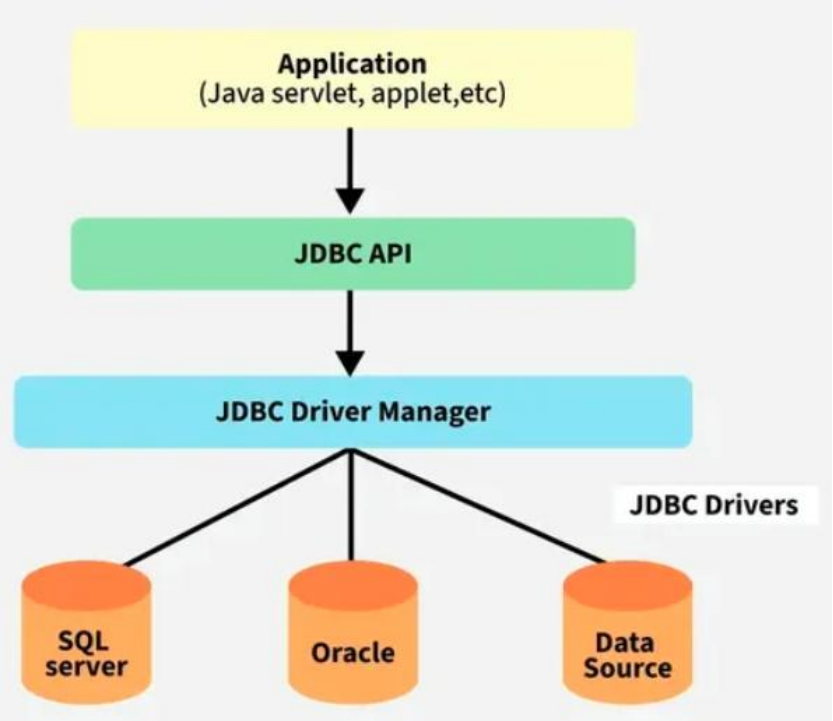
JDBC: JDBC is an API that helps applications to communicate with databases, it allows Java programs to connect to a database, run queries, retrieve, and manipulate data. Because of JDBC, Java applications can easily work with different relational databases like MySQL, Oracle, PostgreSQL, and more.

JDBC Architecture:



1.Application: It can be a Java application or servlet that communicates with a data source.
2.The JDBC API: It allows Java programs to execute SQL queries and get results from the database. Some key components of JDBC API include

- Interfaces like Driver, ResultSet, RowSet, PreparedStatement, and Connection that helps managing different database tasks.
- Classes like DriverManager, Types, Blob, and Clob that helps managing database connections.

3.DriverManager: It plays an important role in the JDBC architecture. It uses some database-specific drivers to effectively connect enterprise applications to databases.
4.JDBC drivers: These drivers handle interactions between the application and the database.

Two-Tier Architecture: A Java Application communicates directly with the database using a JDBC driver. It sends queries to the database and then the result is sent back to the application. For example, in a client/server setup, the user's system acts as a client that communicates with a remote database server.
Structure:
Client Application (Java) -> JDBC Driver -> Database

Three-Tier Architecture
In this, user queries are sent to a middle-tier services, which interacts with the database. The database results are processed by the middle tier and then sent back to the user.
Structure:
Client Application -> Application Server -> JDBC Driver -> Database

Difference table:

Feature	2-Tier Architecture	3-Tier Architecture
Structure	The client application communicates directly with the database using JDBC drivers.	The client communicates with an application server, which then interacts with the database through JDBC.
Security	Less secure, because the database is directly accessible from the client side.	More secure, since the database is hidden behind the server and only the server accesses it.
Scalability	Not very scalable, as every client needs its own connection to the database.	Highly scalable, because the server manages database connections (e.g., via connection pooling).
Complexity	Simple design, easy to implement for small programs.	More complex, as it requires an extra application server layer.
Best suited for	Small-scale applications with few users.	Large, enterprise, or web-based applications needing security and performance.

Types of drivers: JDBC drivers are client-side adapters (installed on the client machine, not on the server) that convert requests from Java programs to a protocol that the DBMS can understand. There are 4 types of JDBC drivers:

1. **Type 1: JDBC-ODBC Bridge Driver**
   - Uses ODBC (Open Database Connectivity) to connect to the database.
   - Requires ODBC driver installed on the client machine.
   - Pros: Easy to use, quick for prototyping.
   - Cons: Slow (two layers), platform dependent, not used in modern apps.
2. **Type 2: Native-API Driver**
   - Uses JDBC calls, converted into database-specific native API (like C/C++) calls.
   - Needs vendor-specific native libraries installed.
   - Pros: Faster than Type 1.
   - Cons: Not portable (depends on OS and DB libraries).
3. **Type 3: Network Protocol Driver**
   - Uses a middleware server (application server) to translate JDBC calls into database protocol.
   - Client ↔ Middleware ↔ Database.
   - Pros: Highly flexible, no need for DB libraries on client.
   - Cons: Adds an extra layer (server required).
4. **Type 4: Thin Driver (Pure Java Driver)**
   - Converts JDBC calls directly into database protocol using pure Java.
   - No native libraries or middleware required.
   - Pros: Fast, portable, widely used today.
   - Cons: Vendor-specific (different for Oracle, MySQL, etc.).

Types of Statement in JDBC: In Java, the Statement interface in JDBC (Java Database Connectivity) is used to create and execute SQL queries in Java applications. JDBC provides three types of statements to interact with the database: **Basic Statement, Prepared Statement, Callable Statement.**

1.Statement: A Statement object is used for general-purpose access to databases and is useful for executing static SQL statements at runtime.

Syntax: Statement statement = connection.createStatement();

Implementation: Once the Statement object is created, there are three ways to execute it.

- **execute(String SQL):** It is used to executes any SQL statements (like SELECT, INSERT, UPDATE or DELETE). If the ResultSet object is retrieved, then it returns true else false is returned.
- **executeUpdate(String SQL):** It is used to executes SQL statements (like INSERT, UPDATE or DELETE). It returns the number of rows affected by the SQL statement.
- **ResultSet executeQuery(String SQL):** It is used to executes the SELECT query. It returns a ResultSet object that contains the data retrieved by the query.

Ex:

```
Statement st = con.createStatement();
ResultSet rs = st.executeQuery("SELECT * FROM students");
```

2.Prepared Statement: A PreparedStatement represents a precompiled SQL statement that can be executed multiple times. It accepts parameterized SQL queries, with ? as placeholders for parameters, which can be set dynamically.

Syntax: PreparedStatement pstmt = con.prepareStatement("SELECT * FROM table_name WHERE column1 = ? AND column2 = ?");

Implementation: Once the PreparedStatement object is created, there are three ways to execute it:

- **execute():** This returns a boolean value and executes a static SQL statement that is present in the prepared statement object.
- **executeQuery():** This returns a ResultSet from the current prepared statement.
- **executeUpdate():** This returns the number of rows affected by the DML statements such as INSERT, DELETE, and more that is present in the current Prepared Statement.

Ex:

```
PreparedStatement ps = con.prepareStatement("SELECT * FROM students WHERE id = ?");
ps.setInt(1, 101);
ResultSet rs = ps.executeQuery();
```

3.Callable Statement: A CallableStatement is used to execute stored procedures in the database. Stored procedures are precompiled SQL statements that can be called with parameters. They are useful for executing complex operations that involve multiple SQL statements.

Syntax: CallableStatement cstmt = con.prepareCall("{call ProcedureName(?, ?)}");

{call ProcedureName(?, ?)}: Calls a stored procedure named ProcedureName with placeholders ? for input parameters.

Methods to Execute:

- **execute():** Executes the stored procedure and returns a boolean indicating whether the result is a ResultSet (true) or an update count (false).
- **executeQuery():** Executes a stored procedure that returns a ResultSet.
- **executeUpdate():** Executes a stored procedure that performs an update and returns the number of rows affected.

Ex:

```
CallableStatement cs = con.prepareCall("{call getStudent(?)}");
cs.setInt(1, 101);
```

ResultSet rs = cs.executeQuery();

Common for all JDBC driver:
Class.forName("com.mysql.jdbc.Driver");
Connection con = DriverManager.getConnection(“jdbc:mysql://localhost/mydb”, “root”, “your_password”);

JDBC Result Set:
1.Navigating a ResultSet:

Method	Description
next()	used for move next row in the ResultSet.
previous()	used for move to previous row in the ResultSet
first()	used for move to first row in the ResultSet
last()	used for move to last row in the ResultSet
absolute(int row)	used to move to specific row
relative(int rows)	used for Moves forward or backward by the specified number of rows
beforeFirst()	used for Positions the cursor before the first row
afterLast()	used for Positions the cursor after the last row

2.Retrieving Data from a ResultSet:

Method	Description
getInt(int columnIndex)	used for Retrieves an integer from the specified column
getString(int columnIndex)	used for Retrieves a string from the specified column
getDouble(int columnIndex)	used for Retrieves a double from the specified column
getBoolean(int columnIndex)	used for Retrieves true or false from the specified column
getDate(int columnIndex)	used for Retrieves a **java.sql.Date**
getObject(int columnIndex)	used for Retrieves any type of object
getArray(int columnIndex)	used for Retrieves a SQL array

3.Updating Data in a ResultSet:

Method	Description
updateInt(int columnIndex, int x)	used for Updates an integer value in the specified column
updateString(int columnIndex, String x)	used for Updates a string value
updateBoolean(int columnIndex, boolean x)	used for Updates a boolean value
updateRow()	used for Updates a row
deleteRow()	used for delete a row

Difference between Statements:

Feature	Statement	PreparedStatement	CallableStatement
Purpose	Executes simple static SQL queries.	Executes precompiled parameterized SQL queries.	Executes stored procedures in the database.
SQL Query	Written directly in code, no parameters.	Query can contain placeholders (?) for values.	Uses {call procedure_name(?,...)} syntax.
Compilation	Compiled every time it runs → slower.	Compiled once, can be reused → faster.	Precompiled and optimized at the database side.
Security	Vulnerable to SQL injection.	Prevents SQL injection (parameters handled safely).	Secure, logic hidden inside stored procedure.
Use Case	One-time, simple queries.	Repeated queries with different inputs.	Complex operations needing procedures/functions.

What is a Java Servlet: A Java Servlet is a server-side Java program that runs inside a web server or application server (like Tomcat or Jetty). It handles client requests (such as HTTP requests), processes them, and generates dynamic responses (like HTML, JSON, or XML). Servlets are a core part of Java EE (Jakarta EE) and are widely used to build efficient, scalable, and dynamic web applications.

Why are Servlets used:

Servlets are used because:

- **Platform independent:** written in Java, they run anywhere.
- Faster and efficient than traditional CGI (Common Gateway Interface).
- **Reusable and maintainable:** built using Java OOP concepts.
- **Secure:** use Java's built-in security features.
- **Integration:** can easily connect with databases, APIs, and other backend systems.

What can a Servlet do:

1.Handle Client Requests: Receive HTTP requests from browsers (form data, URL parameters, etc.).

2.Process Requests: Perform server-side tasks like database access, business logic, and interaction with other components.

3.Generate Dynamic Responses: Send back HTML, JSON, XML, or redirects based on processed data.

4.Server-Side Components: Act as the core logic of Java web applications, managing data flow between client and server.

5.Servlet Container: Run inside containers like Tomcat or Jetty, which manage their lifecycle (loading, initialization, request handling, destruction).

6.In essence: Servlets enable server-side processing and dynamic content generation, making them the backbone of interactive Java web applications.

Servlet API: The Servlet API is a set of interfaces, classes, and methods provided by Java EE (Jakarta EE) that allow developers to create, deploy, and run servlets. It defines how a servlet interacts with the web container (Tomcat, Jetty, etc.) and with clients (like browsers).

1.Main Packages in Servlet API: The Servlet API is mainly found in these packages:

i.jakarta.servlet (or javax.servlet in older versions): Provides core classes & interfaces for servlet development.

ii.jakarta.servlet.http: Provides HTTP-specific classes and interfaces for handling HTTP requests/responses.

2.Core Interfaces in Servlet API:

a)Servlet Interface: The root interface for all servlets. Defines methods for lifecycle:

i.init(): Initialization (called once when servlet loads).

ii.service(): Handles client requests.

iii.destroy(): Cleanup when servlet is removed.

b)ServletRequest Interface: Represents **client request**. Provides methods to get request data like:

i.getParameter(): Read form data / query parameters.

ii.getAttribute(): Read request attributes.

iii.getInputStream(): Read binary data (like file upload).

c)ServletResponse Interface: Represents **server response**. Provides methods to send response to client:

i.getWriter(): Send text/HTML output.

ii.getOutputStream(): Send binary data (images, files, etc.).

iii.setContentType(): Set response type (HTML, JSON, etc.).

d)ServletConfig Interface: Provides **configuration info** for a servlet (like init parameters from web.xml).

e)ServletContext Interface: Represents **application-wide context** shared by all servlets. Useful for: Storing application-level data, Accessing server info, Reading resource files.

3.HTTP-specific Interfaces (from jakarta.servlet.http):

a)HttpServlet Class: Extends the GenericServlet class. Provides HTTP request handling methods:

i.doGet(): Handle GET requests.

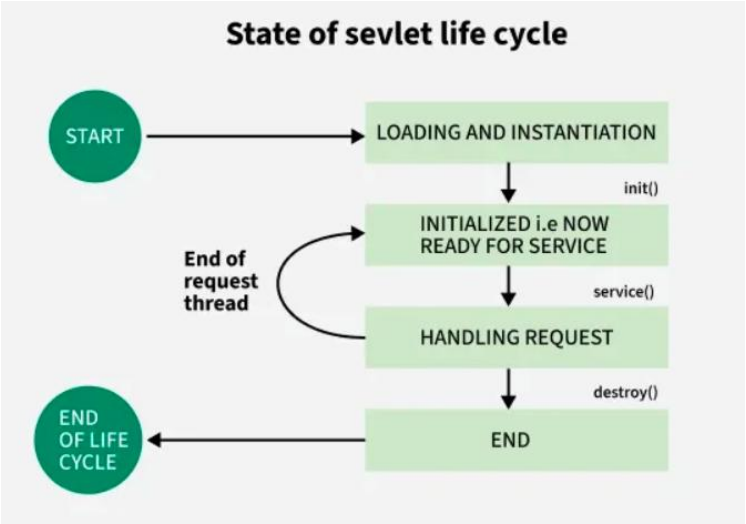
ii.doPost(): Handle POST requests.

iii.doPut(), doDelete(), etc.

b)HttpServletRequest: Extends ServletRequest for HTTP. Provides methods like:

i.getHeader(): Read HTTP headers.
ii.getCookies(): Access cookies.
iii.getSession(): Get session object.
c)HttpServletResponse: Extends ServletResponse for HTTP. Provides methods like:
i.addCookie(): Add cookie to response.
ii.sendRedirect(): Redirect to another page.
iii.setStatus(): Set HTTP status codes (200, 404, 500).
4.Servlet Lifecycle (API in action):
i.Servlet Loading & Initialization: init() (ServletConfig available).
ii.Request Handling: service() → calls doGet(), doPost(), etc. using HttpServletRequest & HttpServletResponse.
iii.Destroy: destroy() before servlet is unloaded.

Life Cycle of Servlet: The life cycle of a servlet is managed by the Servlet Container (e.g., Tomcat, Jetty). It defines the steps from loading the servlet → handling requests → destroying it.



1.Loading & Instantiation: The servlet class is loaded into memory by the container. The container creates an instance of the Servlet using the no-argument constructor. This happens only once when the servlet is first requested (or at server startup if configured with load-on-startup).
2.Initializing a Servlet: After the Servlet is instantiated, the Servlet container initializes it by calling the init(ServletConfig config) method. This method is called only once during the Servlet's life cycle.
3.Handling request: Once the Servlet is initialized, it is ready to handle client requests. The Servlet container performs the following steps for each request:
i.Create Request and Response Objects:

- The container creates ServletRequest and ServletResponse objects.
- For HTTP requests, it creates HttpServletRequest and HttpServletResponse objects.

ii.Invoke the service() Method:

- The container calls the service(ServletRequest req, ServletResponse res) method.
- The service() method determines the type of HTTP request (GET, POST, PUT, DELETE, etc.) and delegates the request to the appropriate method (doGet(), doPost(), etc).

4.Destroying a Servlet: When the Servlet container decides to remove the Servlet, it follows these steps:

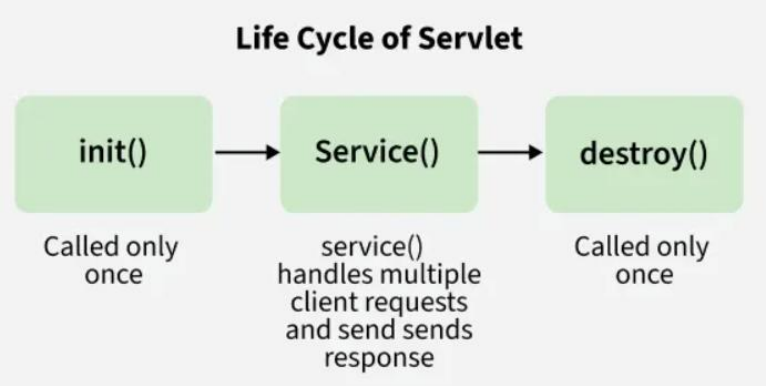
i.Allow Active Threads to Complete: The container ensures that all threads executing the service() method complete their tasks.

ii.Invoke the destroy() Method: The container calls the destroy() method to allow the Servlet to release resources (e.g., closing database connections, freeing memory).

iii.Release Servlet Instance: After the destroy() method is executed, the Servlet container releases all references to the Servlet instance, making it eligible for garbage collection.

Servlet Life Cycle Methods: There are three life cycle methods of a Servlet:

- init()
- service()
- destroy()



1.init() Method: This method is called by the Servlet container to indicate that this Servlet instance is instantiated successfully and is about to put into the service.

Syntax:

```
public void init(ServletConfig config) throws ServletException {
   // Initialization code here
}
```

Use Cases:

i.Open database connections.

ii.Load configuration settings.

iii.Allocate resources.

2.service() Method: This method is used to inform the Servlet about the client requests.

- This method uses ServletRequest object to collect the data requested by the client.
- This method uses ServletResponse object to generate the output content.

Syntax:

```
public void service(ServletRequest req, ServletResponse res)
     throws ServletException, IOException {
   // Request handling code here
}
```

i.Use Cases:

ii.Processing form data.

iii.Communicating with a database.

Sending HTML/JSON/XML response to client.

3.destroy() Method: This method runs only once during the lifetime of a Servlet and signals the end of the Servlet instance.

Syntax:

```
public void destroy() {
  // Cleanup code here
}
```

Use Cases:

i.Close database connections.

ii.Release memory/resources.

iii.Stop background threads.

Simple servlet program:

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;
// A simple Welcome Servlet
public class WelcomeServlet extends HttpServlet {
  // Handles GET requests
  public void doGet(HttpServletRequest request, HttpServletResponse response)
      throws ServletException, IOException {
    // Set response content type
    response.setContentType("text/html");
    // Get the output writer
    PrintWriter out = response.getWriter();
    // Send a simple HTML response
    out.println("<html>");
    out.println("<head><title>Welcome Servlet</title></head>");
    out.println("<body>");
    out.println("<h1>Welcome to Java Servlets!</h1>");
    out.println("</body>");
    out.println("</html>");
  }
}
```

OR

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;
public class WelcomeServlet extends HttpServlet {
  public void doGet(HttpServletRequest request, HttpServletResponse response)
      throws ServletException, IOException {
    // Telling browser the response type is HTML
    response.setContentType("text/html");
    // Writer to send response back to client
    PrintWriter out = response.getWriter();
    // Sending simple HTML page
    out.println("<h1>Welcome to Java Servlets!</h1>");
  }
}
```

Servlet Skeleton: A Servlet Skeleton is the basic template or structure of a servlet program that shows the important lifecycle methods of a servlet. Since servlets are server-side programs, they extend the HttpServlet class to handle client requests and generate responses. The skeleton usually contains the following methods:

1.init(): Called only once when the servlet is first created. It is used for initialization tasks like connecting to databases or loading configuration data.

2.doGet()/doPost(): Called every time a client sends a request.

- doGet() is used to handle HTTP GET requests (like when a user types a URL).
- doPost() is used to handle HTTP POST requests (like form submissions).
    - These methods contain the main logic for processing requests and sending responses.

3.destroy(): Called once when the servlet is being removed from memory. It is used to release resources, close database connections, or perform cleanup tasks.

Skeleton:

```
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import java.io.*;
public class MyServlet extends HttpServlet {
  public void init() throws ServletException {
    // Initialization code
  }
  public void doGet(HttpServletRequest request, HttpServletResponse response)
      throws ServletException, IOException {
    // Code to handle GET requests
  }
  public void doPost(HttpServletRequest request, HttpServletResponse response)
      throws ServletException, IOException {
    // Code to handle POST requests
  }
  public void destroy() {
    // Cleanup code
  }
}
```

What is Swing:

1.Swing is a part of Java Foundation Classes (JFC) used to build Graphical User Interface (GUI) applications in Java.

2.It is a set of lightweight components (like buttons, text fields, tables, menus, etc.) that can be used to create desktop applications.

3.Swing is built on top of Abstract Window Toolkit (AWT) but provides more advanced and flexible components.

Needs of Swing:

1.Limitations of AWT: AWT components are heavyweight (depend on native OS), so they look different on different platforms and have limited functionality.

Example: A button created in AWT looks different in Windows and Linux.

2.Cross-Platform Consistency: Swing applications look and behave the same across all operating systems, unlike AWT.

3.Swing Advantages:

i.Swing components are lightweight: written in pure Java, not dependent on the OS.
ii.Provides rich set of components: tables, trees, tabbed panes, sliders, progress bars, etc.
iii.Customizable Look and Feel (L&F): you can make the GUI look like Windows, Linux, or even custom styles.
iv.Pluggable architecture: components can be extended and customized easily.
v.MVC architecture: separates data (Model), UI (View), and logic (Controller).

JPanel: JPanel is a container component in Swing. It is part of the javax.swing package. A JPanel is basically a blank rectangular area where you can add and arrange other Swing components like buttons, labels, text fields, etc. It acts as a sub-container inside a window (JFrame).

1.JPanel with FlowLayout (Default Layout):

```
import javax.swing.*;
import java.awt.*;
public class JPanelFlowLayout {
  public static void main(String[] args) {
    JFrame frame = new JFrame("JPanel - FlowLayout");
    frame.setSize(400, 200);
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    JPanel panel = new JPanel();
    panel.setBackground(Color.LIGHT_GRAY);
    panel.add(new JButton("Button 1"));
    panel.add(new JButton("Button 2"));
    panel.add(new JButton("Button 3"));
    frame.add(panel);
    frame.setVisible(true);
  }
}
```

2.JPanel with BorderLayout:

```
import javax.swing.*;
import java.awt.*;
public class JPanelBorderLayout {
  public static void main(String[] args) {
    JFrame frame = new JFrame("JPanel - BorderLayout");
    frame.setSize(400, 300);
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    JPanel panel = new JPanel(new BorderLayout()); // use BorderLayout
    panel.setBackground(Color.WHITE);
    panel.add(new JButton("North"), BorderLayout.NORTH);
    panel.add(new JButton("South"), BorderLayout.SOUTH);
    panel.add(new JButton("East"), BorderLayout.EAST);
    panel.add(new JButton("West"), BorderLayout.WEST);
    panel.add(new JButton("Center"), BorderLayout.CENTER);
    frame.add(panel);
    frame.setVisible(true);
  }
}
```

3.JPanel with GridLayout:

```
import javax.swing.*;
import java.awt.*;
public class JPanelGridLayout {
   public static void main(String[] args) {
      JFrame frame = new JFrame("JPanel - GridLayout");
      frame.setSize(300, 200);
      frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
      JPanel panel = new JPanel(new GridLayout(2, 2)); // 2 rows, 2 columns
      panel.setBackground(Color.YELLOW);
      panel.add(new JButton("Button 1"));
      panel.add(new JButton("Button 2"));
      panel.add(new JButton("Button 3"));
      panel.add(new JButton("Button 4"));
      frame.add(panel);
      frame.setVisible(true);
   }
}
```

Component Hierarchy:

1.Object: Root of all Java classes. Every class in Java inherits from Object.

Ex:

```
Object obj = new Object();
System.out.println(obj.toString());
```

2.Component: Abstract class that is the base for all GUI elements.

Ex (AWT component):

```
import java.awt.*;
public class CompExample {
   public static void main(String[] args) {
      Frame f = new Frame("Component Example");
      Button b = new Button("Click Me"); // Button is a Component
      f.add(b);
      f.setSize(200,200);
      f.setVisible(true);
   }
}
```

3.Container: A component that can hold other components.

Ex:

```
Frame f = new Frame("Container Example");
Button b = new Button("OK");
f.add(b);  // Adding button to container
f.setSize(200,200);
f.setVisible(true);
```

4.Window → Frame → JFrame: A Window is a top-level container. Frame is an AWT window. JFrame is the Swing version.

Ex:

```
import javax.swing.*;
public class FrameExample {
```

```
  public static void main(String[] args) {
    JFrame f = new JFrame("JFrame Example");
    JButton b = new JButton("Click Me");
    f.add(b);
    f.setSize(300,200);
    f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    f.setVisible(true);
  }
}
```

5.Panel → Applet → JApplet: Panel is used to group components. Applet/JApplet are small programs that can run inside browsers or applet viewers. (Less used now).

Ex (JPanel):

```
import javax.swing.*;
public class PanelExample {
  public static void main(String[] args) {
    JFrame f = new JFrame("Panel Example");
    JPanel p = new JPanel();
    p.add(new JButton("Inside Panel"));
    f.add(p);
    f.setSize(300,200);
    f.setVisible(true);
  }
}
```

6.JComponent: Base class for all Swing components like buttons, labels, lists.

Ex:

```
JLabel label = new JLabel("I am a JComponent");
```

7.AbstractButton → JButton, JToggleButton, JCheckBox: AbstractButton is parent for all button types. JButton → normal button. JToggleButton → On/Off button. JCheckBox → Checkbox input.

Ex:

```
JButton b = new JButton("Click");
JToggleButton t = new JToggleButton("Toggle");
JCheckBox c = new JCheckBox("Accept Terms");
```

8.JLabel: Used to display text or images.

Ex:

```
JLabel l = new JLabel("Welcome to Swing");
```

9.JComboBox: Dropdown menu for selecting items.

Ex:

```
String[] items = {"Red", "Green", "Blue"};
JComboBox<String> combo = new JComboBox<>(items);
```

10.JList: Displays a list of selectable items.

Ex:

```
String[] data = {"Java", "Python", "C++"};
JList<String> list = new JList<>(data);
```

11.JProgressBar: Shows the progress of a task (loading bar).

Ex:

```
JProgressBar bar = new JProgressBar(0,100);
bar.setValue(50); // Shows 50% progress
```